Course: Enterprise Real-Time Data Engineering
Instructor: Mohamed Fouad Fakhruldeen (Fakhr.me)
Course Portal: fakhr.me/KafkaSpark
Code & Labs Repository: github.com/fakhruldeen/KafkaSpark

Session S04: Hands-On Lab Walkthrough

Consumer Groups Scaling, Rebalance Failover, Manual Commits & DLT Routing

Lab Objectives

In this hands-on terminal lab, you will:
1. Launch multiple consumer instances inside a single `Consumer Group` and observe dynamic partition assignment.
2. Produce keyed records and verify load distribution across active consumer workers.
3. Simulate worker failure and observe real-time Cooperative Rebalancing and partition handover.
4. Implement manual offset commits to guarantee At-Least-Once processing semantics.
5. Seed malformed poison-pill records and verify automated isolation into a Dead Letter Topic (`.DLT`).

Step 1: Pre-Flight Environment Check

Navigate to the lab workspace:

```
cd "${LAB_ROOT:-delivery_materials/00_Lab}" 2>/dev/null || cd 00_Lab || cd ../../00_Lab
tools/check_stack.sh
```

Ensure all 6 services report `PASS` .

Step 2: Create Multi-Partition Topics

Create the main orders topic and the quarantine Dead Letter Topic:

```
docker exec -it kafka1 /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server kafka1:9092 \
  --create --topic nile.retail.orders \
  --partitions 3 --replication-factor 3 \
  --config min.insync.replicas=2

docker exec -it kafka1 /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server kafka1:9092 \
  --create --topic nile.retail.orders.DLT \
  --partitions 3 --replication-factor 3 \
  --config min.insync.replicas=2
```

Step 3: Run Worker A in Terminal 1

Open a terminal window and launch the first consumer instance:

```
python3 -m python.s04_consumer --group nile-warehouse-workers --client-id worker-A
```

Observe the initial assignment:
`Worker worker-A assigned partitions: [0, 1, 2]`

Step 4: Run Worker B in Terminal 2 (Observe Dynamic Rebalance)

Open a second terminal window and launch a second consumer in the exact same group:

```
python3 -m python.s04_consumer --group nile-warehouse-workers --client-id worker-B
```

Watch both terminals simultaneously!

Notice how the coordinator triggers a Cooperative Rebalance:
- `worker-A` yields Partition 2 and retains `[0, 1]`.
- `worker-B` is assigned Partition `[2]`.

Step 5: Produce Live Traffic and Observe Parallel Load Balancing

In a third terminal window, produce 300 business orders:

```
python3 -m python.s03_producer --orders 300
```

Observation:

- Terminal 1 (`worker-A`) processes orders belonging to Partitions 0 and 1.
- Terminal 2 (`worker-B`) processes orders belonging to Partition 2.
- No consumer blocks the other; throughput scales linearly!

Step 6: Simulate Worker Crash (Chaos Failover)

Press `Ctrl + C` in Terminal 1 to kill `worker-A`.
Look immediately at Terminal 2 (`worker-B`):
Within seconds, the coordinator detects `worker-A` 's departure and re-assigns all 3 partitions `[0, 1, 2]` to `worker-B` !
Zero messages are lost, and processing continues uninterrupted.

Step 7: Run Unit Logic Tests for S04

Verify the manual commit and DLT routing logic using pytest:

```
.venv/bin/pytest tests/test_s04_consumer_logic.py -v
```

Step 8: Run the S04 Automated Verification Script

Run the gate validator script:

```
../delivery_materials/S04_Consumer_Internals_Rebalance_Reliability/Trainer/verify_s04.sh
```

Verify that all 5 tests PASS and confirm:
`★ PASS: S04 Lab Requirements 100% Satisfied! ★`

🌟 Gold Stretch Challenge: Simulating Stalled Consumers & Cooperative Sticky Rebalance

Distinguished Engineer Challenge: If you complete the core lab before time expires, simulate a processing timeout incident:

1. Launch two consumers in the same consumer group with an aggressive poll timeout:

- Configure `max.poll.interval.ms=10000` (10 seconds) and insert a 15-second `time.sleep()` in the processing loop.

2. Monitor consumer group behavior with:

`bash`

```
   docker exec -it kafka1 /opt/kafka/bin/kafka-consumer-groups.sh      --bootstrap-server kafka1:9092      --describe --group retail-order-processors --state
```

3. Observe how the Group Coordinator evicts the stalled consumer, marks the group as `PreparingRebalance`, and reassigns partitions using the `CooperativeStickyAssignor` without interrupting unaffected group members.